

Medium Day is this Friday. Don't miss it. [Register for free now!](#)



Rust



Machine Learning



LLM



Open Source



Programming



I Fully Designed an AI Voice Studio in Rust — No Python, No PyTorch. Why Won't I Build It?



Aarambh Dev Hub

Following ▾

13 min read · 10 hours ago



3



Six months ago I sat down and made myself a promise. I was going to build a complete Large Language Model in pure Rust — not a wrapper, not a tutorial toy, not an API client. A real pretraining pipeline, attention kernels, GRPO alignment, Flash Attention — the whole thing. No Python. No PyTorch. No excuses.

That project became [aarambh-studio](#), and it changed how I think about systems engineering.

When it reached its planned end-state at version four — fifty-five phases, four major versions, production-grade training on free Kaggle GPUs — the obvious question appeared. Language models generate tokens. But the world doesn't only communicate through text. The world communicates through **sound**.

That question became aarambh-voice-studio.

And now I've designed it completely. Twenty-four modular Rust crates. A twenty-eight phase build roadmap. Full starter code. Three generative engines capable of synthesizing speech, cloning voices in three seconds, generating instrumental music, and composing full songs with verse-chorus-bridge structure.

I am not actively building it anymore. I am giving the entire blueprint away.

Here is exactly why — and exactly how it works.

The Problem With Audio AI in Python (That Nobody Talks About Honestly)

Search GitHub for “text to speech open source” right now and you’ll find the same pattern repeating itself endlessly. A `requirements.txt` with seventy-five dependencies. Three gigabytes of PyTorch wheels. CUDA driver conflicts. Unmaintained C extensions from 2017. Six gigabytes of RAM consumed before a single millisecond of audio is generated.

In the text LLM world, engineers have tolerated this because model size dwarfs runtime overhead. But audio is categorically different.

Audio is real-time. A fifty millisecond garbage collector pause in Python doesn’t cause a missed word in a chat response — it causes an audible click or pop in a voice stream that the listener immediately notices and judges. Latency and determinism in audio aren’t nice-to-haves. They are the product.

Worse, the entire Python audio AI ecosystem is fragmented. Bark for speech. MusicGen for music. EnCodec for compression. None of them share a common tensor representation. None share a config format. If you want to take an instrumental track from MusicGen, mix it with a singing voice from a separate model, and master it into a stereo FLAC — you are gluing shell scripts together and bouncing temporary files on disk.

aarambh-voice-studio was designed to reject this entire paradigm from the first line of `Cargo.toml`.

Why Rust, and Why Candle

By building on top of Candle — Hugging Face's pure-Rust minimalist tensor framework — the entire studio has zero Python runtime dependency. Memory allocation is deterministic and bound by Rust's ownership model. When an audio buffer goes out of scope, the memory is reclaimed instantly, without a GC pause.

The entire system compiles to a single native binary. No interpreter startup. No dependency hell. On edge devices, serverless containers, or embedded systems — just ship the binary.

Most importantly: voice, music, singing, and mixing are not separate tools bolted together. They are three engines built on top of a **single, unified transformer core** and a **single shared neural audio codec**. A voice token, an instrumental token, and a pitch tracking token all live in the same mathematical latent space, governed by the same Rust type system. That unification is what makes a true studio — not a collection of scripts — possible.

The Foundation: Why 44,100 Samples Per Second Breaks Transformers

To understand the architecture, you first need to understand why audio is fundamentally harder than text for a neural network to model.

A text LLM like aarambh-studio tokenizes an English sentence into twenty or thirty token IDs using Byte Pair Encoding. A model generating five tokens

per second is producing readable text at conversational speed.

Now consider audio. CD-quality sound is sampled at 44,100Hz. One second of audio is 44,100 individual floating-point amplitude values. A ten-second clip is nearly half a million data points. Feed those raw samples into an autoregressive transformer and the sequence length explodes — and since attention scales quadratically with sequence length, even two seconds of audio would bring a high-end GPU to a crawl.

Early systems like WaveNet tried generating one sample at a time at 16kHz. Real-time inference was nearly impossible. Spectrogram-based approaches like HiFi-GAN were faster but discarded phase information, producing metallic artifacts and unnatural robotic timbres.

The modern breakthrough: **Neural Audio Codecs.**

Instead of generating raw waveforms, a codec encoder-decoder network compresses continuous audio into discrete acoustic tokens — and reconstructs perfectly on the other end. Think of it as the audio equivalent of a tokenizer, except the “tokenizer” is a deep neural network trained with perceptual losses to preserve every nuance of human timbre and instrumental character.

The 12.5Hz Codec: The Most Critical Crate in the Entire Workspace

Most neural audio codecs — EnCodec, DAC, SoundStream — operate between 50Hz and 75Hz. That means 50 to 75 acoustic tokens per second. For a ten-

second audio clip, that is 500 to 750 tokens. Already stressful for a transformer backbone.

The architecture decision that separates aarambh-voice-studio from everything else: we compress to **12.5Hz**.

That is twelve and a half tokens per second. A ten-second clip becomes 125 tokens — small enough for a standard transformer running on an Intel Core i3 laptop to process in real time, without a GPU.

This is not magic. It requires a carefully designed codec architecture:

- A **Residual Vector Quantizer (RVQ)** operating across multiple codebooks at progressively finer acoustic resolutions
- A **transformer bottleneck layer** between the convolutional encoder and the RVQ, which compresses rich acoustic features into a more compact semantic representation
- **Semantic distillation** from self-supervised speech representations (trained on Whisper-style contrastive signals), which teaches the codec to preserve linguistic and phonemic information rather than just raw waveform shape at extremely low frame rates

If this codec fails, everything downstream fails. That is why `aarambh-vision-codec` is Phase Zero of the roadmap, and it has the most extensive test suite of any crate in the workspace.

The Three-Engine Architecture

Once the codec foundation is solid, the rest of the studio is built on top of a **shared transformer backbone** (`aarambh-vision-nn`) that is directly adapted from the same decoder-only architecture used in `aarambh-studio`. Here is how the three engines are layered:

Engine One: The Voice Engine

The voice engine (`aarambh-voice-model`) handles three related tasks unified by the same codec token vocabulary:

Text-to-Speech (TTS): The phoneme preparation crate (`aarambh-voice-textprep`) converts raw text through a multilingual Grapheme-to-Phoneme conversion pipeline, producing an international phoneme alphabet representation. This allows accurate pronunciation across English, Hindi, and code-switched multilingual speech without retraining. The transformer generates acoustic tokens that the codec decoder converts to waveforms.

Zero-Shot Voice Cloning: Given as little as three seconds of a speaker's voice, the system extracts a speaker embedding from the codec's latent space and conditions the transformer through cross-attention. The model then generates new speech in that voice without any fine-tuning or per-speaker training. The key insight: because speaker identity is captured in the codec's compressed latent space, you only need a few hundred acoustic tokens — not thousands of raw waveform samples — to characterize a speaker completely.

Continuous Emotional Control: Rather than discrete emotion labels (happy, sad, angry), the voice engine uses an eight-dimensional continuous emotional control space derived from the valence-arousal emotional circumplex model. The conditioning MLP projects this eight-dimensional vector into cross-attention keys and values, giving the model precise, smooth, expressive control over prosody, pitch range, and speaking pace —

the difference between a system that can simulate warmth versus one that actually sounds human.

Engine Two: The Music Engine

The music engine introduces a critical architectural insight that most music AI projects skip entirely: **you cannot generate good music without first understanding music.**

Before the music transformer generates a single token, a dedicated music understanding module (`aarambh-voice-understand`) runs on the input prompt to classify:

- **Genre:** Electronic, orchestral, jazz, folk, ambient
- **Tempo:** Beats-per-minute range
- **Key and Mode:** Major, minor, Dorian, Lydian
- **Mood:** Energetic, melancholic, tense, peaceful
- **Instrumentation Density:** Solo, chamber, ensemble, full production

These classifiers are trained first — in Phase Ten of the roadmap — using auto-labeled datasets. They then serve as conditioners for the music generation transformer, which produces acoustic tokens corresponding to instrumental sounds. The separation of understanding from generation is what prevents the common failure mode where music AI systems produce technically playable but emotionally incoherent output.

Engine Three: The Singing Engine

The singing engine is the most complex subsystem in the entire workspace, for one reason: singing requires the system to simultaneously control phonemes (what words to say), pitch (what melody note to hit), duration (how long to hold each syllable), and breath support timing.

The architecture layers:

1. A **melody pitch tracker** that converts a MIDI sequence or fundamental frequency contour into frame-level pitch conditioning vectors
2. A **lyrics-to-duration aligner** that stretches phoneme sequences to match the musical rhythm using attention-based forced alignment
3. The shared transformer backbone generating codec tokens conditioned on both pitch and phoneme streams simultaneously
4. An optional **diffusion refinement head** — a four-step Rectified Flow Matching pass that post-processes the raw codec tokens to add breath sounds, vibrato, and natural vocal articulation that autoregressive sampling alone rarely captures

The Song Composer: Structure Before Sound

One of the most neglected problems in AI music generation is structure. When you or I listen to a song we love, we don't just hear a continuous stream of pleasant sounds. We hear a narrative arc — an intro that builds tension, a verse that tells a story, a chorus that releases emotion, a bridge that surprises.

Most open-source music AI systems generate audio as one flat continuous sequence. The result sounds like someone improvising endlessly without ever arriving at a point.

The aarambh-voice-studio Song Composer (`aarambh-voice-compose`) separates structure planning from audio generation:

Phase 1 — Structure Planning: Given a prompt like “a melancholic indie folk song about missing home,” the structure planner predicts the full song layout — intro (8 bars), verse 1 (16 bars), chorus (8 bars), verse 2 (16 bars), chorus (8 bars), bridge (8 bars), outro (4 bars). It also predicts per-section instrumentation density and mood targets.

Phase 2 — Stem Generation: Each structural section is generated independently by the music engine and singing engine, conditioned on its section-specific mood and instrumentation plan.

Phase 3 — Multi-Track Mixing: The individual audio stems are mixed using a learned gain and EQ prediction network that balances vocal presence against instrumental density, applies dynamic range compression appropriate to the genre, and produces a final stereo master at the correct loudness target.

This architecture is why the Song Composer is one of the most exciting and underexplored research directions in the entire roadmap. If you want to build one part of this studio and publish a paper, the Structure Planner is likely where the highest-impact contribution lives.

Online Self-Learning: New Voices in Seconds, Zero Forgetting

One of the most practically valuable features of aarambh-voice-studio is its online self-learning subsystem (`aarambh-voice-selflearn`), which borrows architectural principles from our separate Manas continual learning project.

The problem it solves: a user records three seconds of a new speaker's voice and wants the system to remember that voice permanently — not just for the current session, but the next time they open the application, even after the model has been updated with new data in the meantime.

Naive fine-tuning would cause catastrophic forgetting — learning the new voice by overwriting old knowledge.

The aarambh approach:

1. The new speaker recording is passed through the codec encoder, producing a small sequence of high-level speaker embeddings
2. These embeddings are stored in an **indexed associative memory bank** alongside a compact LoRA adapter delta computed from the few-shot examples
3. At inference time, the memory bank performs nearest-neighbor lookup to retrieve the relevant speaker adapter, which is dynamically applied on top of the frozen base model weights
4. New adapter updates are **gradient-orthogonalized** against the subspace spanned by existing stored adapters, ensuring that learning a new voice does not degrade any previously stored voice

The result: unlimited voice registration over time, with no catastrophic forgetting and no need to ever retrain the base model.

Alignment: GRPO and DPO Without a Human Labeler in Sight

Getting audio models to produce genuinely good output — not just technically valid sequences — requires alignment beyond supervised training. `aarambh-voice-studio`'s alignment crate (`aarambh-voice-align`) implements both Group Relative Policy Optimization (GRPO) and Direct Preference Optimization (DPO), adapted for discrete acoustic token sequences.

For GRPO: given a prompt, the model generates a group of G candidate audio outputs with different noise seeds. An automatic reward model scores each candidate on:

- **Speaker similarity:** cosine distance between generated and reference speaker embeddings
- **Intelligibility:** phoneme error rate as measured by a lightweight CTC speech recognizer
- **Naturalness:** a learned MOS (Mean Opinion Score) predictor trained on human preference labels
- **Prosody adherence:** pitch contour correlation with an emotional target

The relative advantage $A_i = (R_i - \text{mean}(R)) / \text{std}(R)$ is computed within the group, and the policy is updated via a clipped surrogate objective — the same mathematical machinery from language model RLHF, but operating over acoustic token policies rather than text token distributions.

No human annotators needed. No reward hacking risks from a single omnipotent reward model. Clean, mathematically principled improvement.

Ethical Engineering: Safety Baked Into the Type System

The safety architecture of `aarambh-voice-studio` (`aarambh-voice-safety`) treats consent and provenance not as policy decisions implemented by runtime checks, but as Rust type-system constraints enforced at compile time.

Every synthesized voice output must carry a **ConsentToken** in its type signature — a cryptographically signed structure that attests the speaker whose voice was used in cloning provided explicit informed consent for this use case. Code that attempts to synthesize audio without a valid **ConsentToken** literally cannot compile. There is no runtime bypass. The type checker enforces it.

Additionally, every audio file produced by the studio has a **psychoacoustic watermark** imperceptibly embedded in the frequency domain. The watermark encodes a SHA-256 hash of the generation session metadata (prompt, speaker identity, timestamp, model version) in frequency bands that survive typical lossy recompression to MP3 or OGG, but remain completely inaudible to human listeners.

This is not ethical theater. These are engineering decisions that make abuse materially harder at the systems level.

The 24-Crate Architecture at a Glance

The full workspace spans twenty-four library crates plus one CLI binary:

Layer	Crate	Responsibility
L0	<code>`aarambh-voice-core`</code>	Types, config, errors, SafeTensors I/O
L0	<code>`aarambh-voice-kernel`</code>	Low-level DSP primitives, resampling
L1	<code>`aarambh-voice-codec`</code>	Neural audio codec – RVQ encoder/decoder
L1	<code>`aarambh-voice-textprep`</code>	G2P, phoneme sequences, multilingual
L2	<code>`aarambh-voice-data`</code>	Dataset loading, augmentation, bucketing
L2	<code>`aarambh-voice-understand`</code>	Music understanding classifiers
L2	<code>`aarambh-voice-nn`</code>	Transformer primitives, RoPE, SwiGLU
L3	<code>`aarambh-voice-model`</code>	Three-engine model definition
L3	<code>`aarambh-voice-weights`</code>	Weight loading, conversion, checkpointing
L4	<code>`aarambh-voice-train`</code>	Training loop, loss functions
L4	<code>`aarambh-voice-quant`</code>	GGUF/GPTQ quantization
L5	<code>`aarambh-voice-finetune`</code>	LoRA, adapter, voice personalization
L5	<code>`aarambh-voice-align`</code>	GRPO + DPO acoustic alignment
L5	<code>`aarambh-voice-selflearn`</code>	Associative memory, gradient orthogonalization
L6	<code>`aarambh-voice-dsp`</code>	Mastering chain: EQ, compression, LUFS
L6	<code>`aarambh-voice-compose`</code>	Song Structure Planner + stem mixing
L7	<code>`aarambh-voice-safety`</code>	Consent tokens + psychoacoustic watermarking
L7	<code>`aarambh-voice-eval`</code>	MOS, PER, UTMOS, speaker similarity
L8	<code>`aarambh-voice-inference`</code>	Streaming generation, speculative decoding
L8	<code>`aarambh-voice-serve`</code>	Axum REST API, WebSocket streaming
L9	<code>`aarambh-voice-studio`</code>	CLI binary – the entry point

Four model scales designed for commodity hardware:

- **Tiny (~10M params):** Verifies on CPU, Intel Core i3, no GPU needed
- **Small (~55M params):** Trains on free Kaggle T4 GPU tier in days
- **Medium (~170M params):** Full production quality on Kaggle P100
- **Large (~450M params):** High-fidelity synthesis for studio mastering

The 28-Phase Roadmap: How to Actually Build This

The repository includes a detailed twenty-eight phase roadmap, with each phase defining exact engineering goals, file-by-file task checklists, automated tests you can run to verify milestone completion, and a single command milestone test that proves the phase is done.

The first seven phases build the Mini 7-Crate TTS Variant — a stripped-down build path targeting a working zero-shot voice cloning engine in one to two weeks. This is designed for engineers who want to contribute meaningfully without committing to the full twenty-four crate vision immediately.

Phases Eight through Twenty-Eight complete the full studio: music understanding, music generation, singing synthesis, the song composer, GRPO alignment, online self-learning, and production-grade inference

Open in app ↗



Medium



Search



Write



tells you whether it passed.

Why I'm Giving It Away

The honest answer is the same one I gave at the end of aarambh-studio: I think the most valuable thing I can do with these architectural designs is make them completely free and available to people who have the time, skills, and motivation to actually build them out.

I spent months designing this system properly — not rushing to a demo, but thinking through every architectural decision, every failure mode, every interaction between subsystems. The codec design. The emotional control space. The song structure planner. The safety type system. Every piece of this was designed to be correct before it was designed to be fast.

But designing a system and building a system are two different projects. And I have another system calling for my attention.

The open-source community — particularly the growing ecosystem of Rust engineers who are serious about systems programming for AI — is better positioned than I am to take these blueprints and realize them. That is not a resignation. That is a deliberate decision about where leverage lives.

The repository is ready. The roadmap is complete. The starter code is in place.

If you want to build an AI audio studio in pure Rust from first principles — one that actually understands music, actually remembers voices, and actually treats safety as a systems concern rather than an afterthought — this is your starting point.

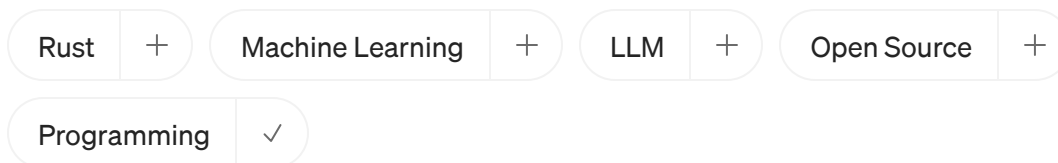
Resources

- **GitHub:** github.com/AarambhDevHub/aarambh-voice-studio
- **YouTube Walkthrough:** [I Fully Designed an AI Voice Studio in Rust — No Python, No PyTorch](https://www.youtube.com/watch?v=8f702b727809)
- **Discord Community:** discord.gg/HDth6PfCnp

- **LinkedIn:** [Darshan Vichhi — Rust Developer](#)

Aarambh Dev Hub engineers real AI systems from first principles in pure Rust. No Python. No PyTorch. No compromises.

If this post was useful, clapping helps other engineers find it. And if you are planning to fork and build — I genuinely want to hear which part you are starting with.



Written by Aarambh Dev Hub

274 followers · 1 following

Following ▾

Rust developer sharing coding tutorials, backend tips, and insights. Follow for deep dives into Rust, programming challenges, and modern development practices.

No responses yet



Alex Mylnikov

What are your thoughts?

